

Documentación Hito 6

1. Patrones de consultas para cassandra

- “Quiero ver los últimos comentarios de un partido realizados por usuarios de mi país.”
- “Quiero ver el último comentario creado para el partido en este país”
- “Quiero ver el último comentario mas popular de este partido realizado en este país”
- “¿Cuáles son los comentarios realizados por un usuario determinado?”
- "Dame los comentarios argentinos de ARG-FRA entre las 18:30 y las 18:40"

2. Definición de particionamiento

En nuestro modelo, las personas estarán comentando constantemente sobre lo que está pasando en un partido, con aun mayor frecuencia que la de un vivo estándar. Para mantener performance se sugiere que espectadores de un partido, comenten y vean principalmente aquellos comentarios de su propio país, tendría poco que un argentino viendo ARG-ING vea comentarios de un país en un idioma que no comprende (por ejemplo, que estuviese viendo comentarios de alemania), asimismo, dada la cantidad por hora, es importante fragmentarlos por periodo horario. Para lo anterior, definimos nuestra primary key así:

```
PRIMARY KEY (  
    (partido_id, pais, ventana),  
    creado_en,  
    comentario_id  
)
```

quedandonos:

```
CREATE TABLE comentarios_por_partido_pais (  
    partido_id text,  
    pais text,  
    ventana timestamp,  
    creado_en timestamp,  
    comentario_id uuid,  
    usuario_id text,  
    usuario_nombre text,  
    contenido text,  
    likes int,  
    estado_moderacion text,  
  
    PRIMARY KEY (  
        (partido_id, pais, ventana),  
        creado_en,  
        comentario_id  
    )  
)
```

```
)
WITH CLUSTERING ORDER BY (
  creado_en DESC,
  comentario_id ASC
);
```

Utilizamos partido porque debemos saber específicamente cual es, no tendría sentido seguir trabajando sobre una partición que contiene datos de partidos ya finalizados, asimismo registrar el país al que corresponde el partido permite evitar mezclar distintos países, los cuales asumimos no tienen intención de comunicarse ni tampoco la capacidad para hacerlo. El beneficio principal de usar una ventana temporal es evitar que todos los comentarios de un partido y país terminen acumulados en una única partición enorme. Con una ventana de 30 minutos, estimamos que una partición debería mantenerse aproximadamente en:

- 100.000 a 200.000 comentarios
- alrededor de 30–60 MB, suponiendo comentarios y metadatos de tamaño moderado
- unas 50–100 escrituras por segundo por partición en un escenario de alta actividad

Realizamos una inserción

```
INSERT INTO comentarios_por_partido_pais (
  partido_id,
  pais,
  ventana,
  creado_en,
  comentario_id,
  usuario_id,
  usuario_nombre,
  contenido,
  likes,
  estado_moderacion
)
VALUES (
  'ARG-FRA',
  'ARG',
  '2030-06-15 18:30:00',
  '2030-06-15 18:42:10',
  uuid(),
  'USR001',
  'Facundo',
  'Vamos Argentina!',
  12,
  'PUBLICADO'
);
```

Asimismo reconocemos que es importante tener un acceso secundario, en especial para manejar otro tipo de consultas como:

- “¿Cuál es el comentario con más likes?”

Por lo tanto creamos una tabla específicamente diseñada para trabajar sobre popularidad, definiendo:

```
CREATE TABLE comentarios_por_likes_partido_pais (  
  partido_id text,  
  pais text,  
  ventana timestamp,  
  
  likes int,  
  creado_en timestamp,  
  comentario_id uuid,  
  
  usuario_nombre text,  
  contenido text,  
  
  PRIMARY KEY (  
    (partido_id, pais, ventana),  
    likes,  
    creado_en,  
    comentario_id  
  )  
)  
WITH CLUSTERING ORDER BY (  
  likes DESC,  
  creado_en DESC,  
  comentario_id ASC  
);
```

Si el módulo debiera soportar reportes analíticos históricos, sería necesario incorporar tablas o estructuras específicas para agregaciones y consultas de largo plazo, ya que la tabla principal está optimizada para accesos operativos por partido, país y ventana temporal. La opción sugerida sería, una vez finalizados los partidos, archivarlos en otra tabla `comentarios_historicos_por_partido_pais` manteniendo la partición por periodos de 30mins para evitar una partición gigantesca, viendose así:

```
CREATE TABLE comentarios_historicos_por_partido_pais (  
  partido_id text,  
  pais text,  
  ventana timestamp,  
  
  creado_en timestamp,  
  comentario_id uuid,  
  
  usuario_nombre text,  
  contenido text,  
  likes int,  
  estado_moderacion text,  
  
  PRIMARY KEY (  
    (partido_id, pais, ventana),
```

```
        creado_en,  
        comentario_id  
    )  
)  
WITH CLUSTERING ORDER BY (  
    creado_en DESC,  
    comentario_id ASC  
);
```

Los comentarios de la **tabla operativa** pueden expirar una vez finalizado el partido y después de haber sido copiados correctamente a la tabla histórica. El costo de tombstones se acepta de forma acotada y controlada. Tanto DELETE como TTL generan tombstones en Cassandra, por lo que evitaríamos eliminaciones masivas o frecuentes durante el partido. La limpieza se realizaría una vez archivados los comentarios y preferentemente por ventanas completas, permitiendo que Cassandra los procese posteriormente durante la compactación

3. Resolución de consultas

Una vez creada la tabla resolvemos las consultas definidas anteriormente:

Últimos comentarios de un partido realizados por usuarios de un país

```
SELECT *  
FROM comentarios_por_partido_pais  
WHERE partido_id = 'ARG-FRA'  
    AND pais = 'ARG'  
    AND ventana = '2030-06-15 18:30:00';
```

Último comentario realizado en esa ventana

```
SELECT *  
FROM comentarios_por_partido_pais  
WHERE partido_id = 'ARG-FRA'  
    AND pais = 'ARG'  
    AND ventana = '2030-06-15 18:30:00'  
LIMIT 1;
```

Comentarios realizados dentro de un rango horario

```
SELECT *  
FROM comentarios_por_partido_pais  
WHERE partido_id = 'ARG-FRA'  
    AND pais = 'ARG'  
    AND ventana = '2030-06-15 18:30:00'  
    AND creado_en >= '2030-06-15 18:40:00'  
    AND creado_en < '2030-06-15 18:50:00';
```

Último comentario creado para el partido en este país

```
SELECT *  
FROM comentarios_por_partido_pais  
WHERE partido_id = 'ARG-FRA'  
  AND pais = 'ARG'  
  AND ventana = '2030-06-15 18:30:00'  
LIMIT 1;
```